

Lab 1 Problems & Solutions

Personal Information Display

Question Description:

Personal Information Display Write a Java program that declares variables to store your name (String), age (int), and favorite character (char). Print these values in a formatted manner: Name: [Your Name] Age: [Your Age] Favorite Character: [Char]

Java Implementation:

Main.java

```
/*
Problem 1: Personal Information Display Write a Java program that declara
your name (String), age (int), and favorite character (char). Print the
Name: [Your Name]
Age: [Your Age]
Favorite Character: [Char]
*/

import java.util.Scanner;
public class Main{
    public static void main(String[] args){

        // Problem 1
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = input.next();

        System.out.print("Enter your age: ");
        byte age = input.nextByte();

        System.out.print("Enter your favorite character: ");
        char favChar = input.next().charAt(0);

        System.out.println(" ---- INFORMATION ----");
        System.out.println("Name: "+ name);
        System.out.println("Age: "+ age);
        System.out.println("Favorite Character: "+ favChar);
    }
}
```

Command Line Calculator

Question Description:

Command Line Calculator Write a program that accepts two integers and one operator (+, -, *, /) as command line arguments (String[] args). Perform the operation and display the result. Handle division by zero with an appropriate message.

Java Implementation:

Main.java

```
/*
Problem 2: Command Line Calculator Write a program that accepts two int
operator (+, -, *, /) as command line arguments (String[] args). Perform
the result. Handle division by zero with an appropriate message.
*/

public class Main {
    public static void main(String[] args){
        // Ensure that user enters 3 Values
        if(args.length < 3){
            System.out.println("Usage: java Calculator <num1> <operator>");
            return;
        }

        System.out.println("Arg 0: " + args[0]);
        System.out.println("Arg 1: " + args[1]);
        System.out.println("Arg 2: " + args[2]);

        // Convert inputs from text to numbers
        float n1 = Float.parseFloat(args[0]);
        String symbol = args[1];
        float n2 = Float.parseFloat(args[2]);

        double result = 0;

        switch(symbol) {
            case "+":
                result = n1 + n2;
                break;
            case "-":
                result = n1 - n2;
                break;
            case "x": // As * not works in Terminal
                result = n1 * n2;
                break;
            case "/":
                if (n2 == 0)
                    System.out.println("Cannot Divide by Zero");
                else
                    result = n1 / n2;
                break;
            case "%":
                if (n2 == 0)
```

```

        System.out.println("Cannot divide by Zero");
    else
        result = n1 % n2;
    break;
default:
    System.out.println("Invalid Operation");
    break;
}

System.out.println(n1 + " " + symbol + " " + n2 + " = " + result);
}
}

```

TempConverter

Question Description:

Temperature Converter Write a program to convert Celsius to Fahrenheit using the formula: $F = (C \times 9/5) + 32$. Accept temperature as a command line argument. Use `Double.parseDouble()` to convert the argument.

Java Implementation:

Main.java

```

/*
    Problem 3: Temperature Converter Write a program to convert Celsius
    formula:  $F = (C \times 9/5) + 32$ . Accept temperature as a command line a
    Double.parseDouble() to convert the argument.
*/
public class Main {
    public static void main(String[] args){ // Command Line Argument
        // Must be Run from the CMD or Terminal (Alt + F12)
        // Commands:-
        // 1. javac Main.java
        // 2. java Main [Value]
        // e.g. java Main 25

        // args[0] : First value the user enters it after file name

        double celsius = Double.parseDouble(args[0]);

        double fahrenheit = (celsius * 9 / 5.0) + 32;

        System.out.println("Temperature(C) = " + celsius);
        System.out.println("Temperature(F) = " + fahrenheit );
    }
}

```

Student Marks Analyzer

Question Description:

Student Marks Analyzer Create variables for marks in 5 subjects (byte data type to save memory). Calculate and display: □ Total marks □ Average (as float) □ Percentage (as double)

Java Implementation:

Main.java

```
/*Problem 4: Student Marks Analyzer Create variables for marks in 5 sub
save memory). Calculate and display:
    Total marks
    Average (as float)
    Percentage (as double)
*/

public class Main {
    public static void main(String[] args){
        byte mark1 = 100, mark2 = 85, mark3 = 95, mark4 = 80, mark5 = 9
        byte numberOfSubjects = 5;
        short maxMarkPerSubject = 100;

        int sum = 0;
        float avg = 0;
        double percentage = 0;

        sum = mark1 + mark2 + mark3 + mark4 + mark5;
        avg = sum / (float) (numberOfSubjects);
        percentage = sum / (double) (numberOfSubjects * maxMarkPerSubjec

        System.out.println("Total Sum = " + sum);
        System.out.println("Average = " + avg);
        System.out.println("Percentage% = " + percentage + " %");

    }
}
```

ASCII Value Printer

Question Description:

ASCII Value Printer Write a program that takes a character input (using command line arguments) and prints its ASCII value. Then print the next 5 consecutive characters and their ASCII values. SECTION B: OPERATORS & CALCULATIONS (Questions 6-11)

Java Implementation:

Main.java

```
/*
```

Problem 5: ASCII Value Printer Write a program that takes a character i arguments) and prints its ASCII value. Then print the next 5 consecutive values.

```
*/
public class Main {
    public static void main(String[] args){
        if (args.length == 0) {
            System.out.println("Please provide a character!");
            return;
        }

        // 1. Convert the first character of the String to a char type
        char symbol = args[0].charAt(0);

        // 2. Print the base character and its value in ASCII
        System.out.println("Symbol: " + symbol + " | ASCII: " + (int)sy
        System.out.println("-----");

        // 3. Print the following 5 letters
        for(int i =1; i<=5; i++){
            System.out.println("Next Character: " + (char)(symbol + i)
        }
    }
}
```

Simple Interest Calculator

Question Description:

Simple Interest Calculator Calculate simple interest using the formula: $SI = (P \times R \times T) / 100$. Accept Principal (long), Rate (float), and Time (int) using appropriate data types to minimize memory usage. Display the result with 2 decimal places.

Java Implementation:

Main.java

```
/*
Problem 6: (((Simple Interest))) Calculator Calculate simple interest u
T) / 100. Accept Principal (long), Rate (float), and Time (int) using a
minimize memory usage. Display the result with 2 decimal places.
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        // وضع المبلغ الاساسي
        System.out.print("Enter the Principal Value: ");
    }
}
```

```

        long principal = input.nextLong();

        // وضع نسبة الفائدة
        System.out.print("Enter the Rate Value: ");
        float rate = input.nextFloat();

        // وضع مدة الفائدة
        System.out.print("Enter the Duration(Years): ");
        int time = input.nextInt();

        // حساب الفائدة كمبلغ
        double interest = (principal * rate * time) / 100.0d;

        System.out.println("-----");
        System.out.printf("The Interest Value: %.2f\n", interest);

        // حساب المبلغ الكلي بعد اضافة الفائدة
        System.out.printf("The Whole Balance (after interest): %.2f\n",

    }
}

```

Bitwise Swap

Question Description:

Bitwise Swap Write a program to swap two integers without using a temporary variable or arithmetic operators. Use only bitwise XOR (^) operator. Display values before and after swapping.

Java Implementation:

Main.java

```

/*
Problem 7: Bitwise Swap Write a program to swap two integers without us
variable or arithmetic operators. Use only bitwise XOR (^) operator. Di
swapping.
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.println(" ----- Swapping Numbers ----- ");

        System.out.print("Enter First Number: ");
        int n1 = input.nextInt();

```

```

        System.out.print("Enter Second Number: ");
        int n2 = input.nextInt();

        System.out.println("Before Swap: n1 = " + n1 + ", n2 = " + n2);
//        // Let: n1 = 5 , n2 = 3

//        int result = n1 ^ n2;        // To Save the result with original
//        n1 = result ^ n1;            // (3 ^ 5) ^ 3 = (6 ^ 3) = 5 -> n1
//        n2 = result ^ n2;            // (3 ^ 5) ^ 5 = (6 ^ 5) = 3 -> n2

        // Another Solution (Without Third Variable `result`)
        n1 = n1 ^ n2;                    // (5 ^ 3) = 6 -> n1 = 6
        n2 = n1 ^ n2;                    // (6 ^ 3) = 5 -> So n2 has the val
        n1 = n1 ^ n2;                    // (6 ^ 5) = 3 -> We use the origin

        System.out.println("After Swap:  n1 = " + n1 + ", n2 = " + n2);
    }
}

```

Compound Assignment Demonstrator

Question Description:

Compound Assignment Demonstrator Initialize an integer variable num = 100. Perform the following operations in sequence using compound assignment operators only (+ =, -=, *=, /=, %=): ☐ Add 50 ☐ Subtract 30 ☐ Multiply by 2 ☐ Divide by 5 ☐ Modulus by 7 ☐ Print the value after each operation.

Java Implementation:

Main.java

```

/*
Problem 8: Compound Assignment Demonstrator Initialize an integer variable num = 100.
Perform the following operations in sequence using compound assignment operators
(*=, /=, %=):
    Add50
    Subtract 30
    Multiply by 2
    Divide by 5
    Modulus by 7
    Print the value after each operation.
*/

public class Main {
    public static void main(String[] args){
        int num = 100;
        System.out.println("Initial Value: " + num);

        // num += 50 ----> num = num + 50 ----> 150
        num += 50;
    }
}

```

```

        System.out.printf("Number Value after (+=): %d\n", num);

        // num -= 30 ----> num = num - 30 ----> 120
        num -= 30;
        System.out.printf("Number Value after (-=): %d\n", num);

        // num *= 2 ----> num = num * 2 ----> 240
        num *= 2;
        System.out.printf("Number Value after (*=): %d\n", num);

        // num /= 5 ----> num = num / 5 ----> 48
        num /= 5;
        System.out.printf("Number Value after (/=): %d\n", num);

        // num %= 7 ----> num = num % 7 ----> 6
        num %= 7;
        System.out.printf("Number Value after (%%=): %d\n", num);

        System.out.printf("Final Value of Num: %d", num);
    }
}

```

Circle Measurements

Question Description:

Circle Measurements Given the radius as a double, calculate and display: ☐ Area (πr^2) - use 3.14159 ☐ Circumference ($2\pi r$) ☐ Diameter ($2r$) ☐ Use the ternary operator to check if radius is positive; if not, print "Invalid radius".
 Problem

Java Implementation:

Main.java

```

/*
Problem 9: Circle Measurements Given the radius as a double, calculate
Area( $\pi r^2$ ) - use 3.14159
Circumference ( $2\pi r$ )
Diameter ( $2r$ )
Use the ternary operator to check if radius is positive; if not,
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        // final float PI = 3.14159f;          // Constant Variable or Use

        System.out.print("Enter the Radius: ");
    }
}

```



```

double radius = input.nextDouble();

String result = (radius > 0) ? "Calculations:" : "Invalid radiu
System.out.println(result);

if(radius > 0){
    System.out.printf("Area ( $\pi r^2$ ): %.5f%n", ( Math.PI * Math.po
    System.out.printf("Circumference ( $2\pi r$ ): %.5f%n", ( 2 * Math
    System.out.printf("Diameter ( $2r$ ): %.1f%n", ( 2 * radius ) )
}
}
}

```

Increment-Decrement Sequence

Question Description:

Problem 10: Increment/Decrement Sequence Initialize int a = 5 and int b = 10. Write a program that prints the following sequence using pre/post increment/decrement operators: □ Print a++ (then print current a) □ Print ++b (then print current b) □ Print --a + b-- (then print both values) Problem

Java Implementation:

Main.java

```

/*
Problem 10: Increment/Decrement Sequence Initialize int a = 5 and i
that prints the following sequence using pre-post increment/decreme
Print a++ (then print current a)
Print ++b (then print current b)
Print--a + b-- (then print both values)
*/

public class Main {
    public static void main(String[] args){
        int a = 5, b = 10;

        // Post-Increment
        System.out.println(a++);    // Display original value (5) then
        System.out.println(a);      // 6

        // Pre-Increment
        System.out.println(++b);    // Increment b by 1 -> 10 + 1 = 11
        System.out.println(b);      // 11

        System.out.println("-----");

        // Pre-Decrement
        // System.out.println(--a);    // Decrement a by 1 -> 5 - 1 = 4
        // System.out.println(a);    // 4
        //

```

```
//          // Post-Decrement
//          System.out.println(b--);      // Display the value of b (11) the
//          System.out.println(b);        // 10

        System.out.println(--a + b--);    // 5 + 11 = 16

    }
}
```

Logical Gate Simulator

Question Description:

m 11: Logical Gate Simulator Accept three boolean values (true/false) as command line arguments. Simulate an AND gate, OR gate, and NOT gate using logical operators. Display the truth table results for combinations of these inputs. SECTION C: DECISION MAKING (Questions 12-19) Problem

Java Implementation:

Main.java

```
/*
Problem 11: Logical Gate Simulator Accept three boolean values (true/false) as
command line arguments. Simulate an AND gate, OR gate, and NOT gate using logical
operators. Display the truth table results for combinations of these inputs

-----

A clever tidbit about Boolean.parseBoolean:
- This Java method is very forgiving;
- if you write true (in any case, like TrUe), it will return true
- Any other text in the universe (like "hello" or "1") will aut
  and will not throw an error.
*/

public class Main {
    public static void main(String[] args){
        if(args.length < 3){
            // 1. javac main.java
            // 2. java Main [] [] []
            System.out.println("Usage: java Main <true/false> <true/false> <true/false>");
            return;
        }

        boolean a = Boolean.parseBoolean(args[0]);
        boolean b = Boolean.parseBoolean(args[1]);
        boolean c = Boolean.parseBoolean(args[2]);

        System.out.println("Inputs: A=" + a + ", B=" + b + ", C=" + c);
        System.out.println("-----");
    }
}
```

```

        System.out.println("----- AND Truth Table -----");
        System.out.println("A && B = " + a + " && " + b + " = " + (a && b));
        System.out.println("A && C = " + a + " && " + c + " = " + (a && c));
        System.out.println("C && B = " + c + " && " + b + " = " + (c && b));

        System.out.println("----- OR Truth Table -----");
        System.out.println("A || B = " + a + " || " + b + " = " + (a || b));
        System.out.println("A || C = " + a + " || " + c + " = " + (a || c));
        System.out.println("C || B = " + c + " || " + b + " = " + (c || b));

        System.out.println("----- NOT Truth Table -----");
        System.out.println("Not A = Not " + a + " = " + !a);
        System.out.println("Not A = Not " + b + " = " + !b);
        System.out.println("Not A = Not " + c + " = " + !c);

        System.out.println("NAND Gate !(A && B): " + !(a && b));
        System.out.println("Complex Logic (A && B) || C: " + ((a && b) || c));
    }
}

```

Grade Calculator

Question Description:

m 12: Grade Calculator Accept a percentage (0-100) as input. Use if-else-if ladder to determine the grade: ☐ 90-100: A ☐ 80-89: B ☐ 70-79: C ☐ 60-69: D ☐ Below 60: F ☐ Validate input (reject negative numbers or >100). Problem

Java Implementation:

Main.java

```

import java.util.Scanner;

/*
    Problem 12: Grade Calculator Accept a percentage (0-100) as input.
    determine the grade:
        90-100: A
        80-89: B
        70-79: C
        60-69: D
        Below60: F
        Validate input (reject negative numbers or >100).
*/
public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your Percentage(%): ");
        int percentage = input.nextInt();
    }
}

```

```

        if (percentage < 0 || percentage > 100) {
            System.out.println("Invalid Percentage! Please enter a valu
        }
        else {
            char grade;
            if (percentage >= 90)
                grade = 'A';
            else if (percentage >= 80)
                grade = 'B';
            else if (percentage >= 70)
                grade = 'C';
            else if (percentage >= 60)
                grade = 'D';
            else
                grade = 'F';

            System.out.println("Grade = " + grade);
        }
    }
}

```

Leap Year Checker

Question Description:

m 13: Leap Year Checker Accept a year as input. Determine if it's a leap year using these rules: ☐ Divisible by 4, but not by 100, unless also divisible by 400 ☐ Use nested if statements and logical operators (&&, ||). Problem

Java Implementation:

Main.java

```

/*
Problem 13: Leap Year Checker
    Accept a year as input.
    Determine if it's a leap year using these rules:
        Divisible by 4, but not by 100, unless also divisible by 400
        Use nested if statements and logical operators (&&, ||).

** A leap year occurs every four years.
    It has 366 days instead of 365, with an extra day added on Febr
    This extra day is known as "intercalary day."
    In Arabic, "leap year" translates to "سنة كبيسة" (leap year).

*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){

```

```

Scanner input = new Scanner(System.in);

System.out.print("Enter the Year: ");
int year = input.nextInt();

//      if( ( (year % 100 != 0) && (year % 4 == 0) ) || (year % 400
//          System.out.println(year + " is a Leap Year");
//      }
//      else{
//          System.out.println(year + " is NOT a Leap Year");
//      }

// -----> Nested-IF
if(year % 4 == 0){
    if(year % 100 == 0){
        if(year % 400 == 0){
            System.out.println(year + " is a Leap Year");
        }
        else{
            System.out.println(year + " is not a Leap Year");
        }
    }else{
        System.out.println(year + " is a Leap Year");
    }
}else{
    System.out.println(year + " is not a Leap Year");
}
}
}

```

Number to Words

Question Description:

m 14: Number to Words (Switch) Accept a single digit (0-9) as input. Use a switch statement (traditional or modern arrow syntax) to print the number in words (e.g., input 5 prints "Five"). Include a default case for invalid inputs. Problem

Java Implementation:

Main.java

```

import java.util.Scanner;

/*
Problem 14: Number to Words (Switch) Accept a single digit (0-9) as input. Use a switch statement (traditional or modern arrow syntax) to print the number in words (e.g., input 5 prints "Five"). Include a default case for invalid inputs.
*/
public class Main {
    public static void main(String[] args){

```

```

Scanner input = new Scanner(System.in);

System.out.print("Enter a Single Digit(0-9): ");
byte digit = input.nextByte();

// Traditional Syntax
switch (digit){
    case 0:
        System.out.println("Zero");
        break;
    case 1:
        System.out.println("One");
        break;
    case 2:
        System.out.println("Two");
        break;
    case 3:
        System.out.println("Three");
        break;
    case 4:
        System.out.println("Four");
        break;
    case 5:
        System.out.println("Five");
        break;
    case 6:
        System.out.println("Six");
        break;
    case 7:
        System.out.println("Seven");
        break;
    case 8:
        System.out.println("Eight");
        break;
    case 9:
        System.out.println("Nine");
        break;
    default:
        System.out.println("Invalid Input!");
        break;
}

// Modern-Syntax(Java >= 14)  -> Removes Colon : and break stat
switch(digit){
    case 0 -> System.out.println("Zero");
    case 1 -> System.out.println("One");
    case 2 -> System.out.println("Two");
    case 3 -> System.out.println("Three");
    case 4 -> System.out.println("Four");
    case 5 -> System.out.println("Five");
    case 6 -> System.out.println("Six");
    case 7 -> System.out.println("Seven");
    case 8 -> System.out.println("Eight");

```

```

        case 9 -> System.out.println("Nine");
        default -> System.out.println("Invalid Input!");
    }

    // In Modern Syntax, you can use Switch to directly assign a variable
    String word = switch (digit){
        case 0 -> "Zero";
        case 1 -> "One";
        case 2 -> "Two";
        case 3 -> "Three";
        case 4 -> "Four";
        case 5 -> "Five";
        case 6 -> "Six";
        case 7 -> "Seven";
        case 8 -> "Eight";
        case 9 -> "Nine";
        default -> "Invalid Input";
    };
    // Don't Forget the semi-colon(;) at the end of assignment.

    System.out.printf("%d -> %s\n", digit, word);

    // Another Solution with Array
    String[] words = {"Zero", "One", "Two", "Three", "Four", "Five"};
    if(digit >= 0 && digit <= 9)
        System.out.println("The Word: " + words[digit]);
    else
        System.out.println("Invalid Input");
}
}

```

Menu-Driven Calculator

Question Description:

m 15: Menu-Driven Calculator Display a menu:

1.

Addition 2. Subtraction 3. Multiplication 4. Division 5. Modulus 6. Exit 7. Use a switch-case structure. For choice 1-5, accept two numbers and perform the operation. Loop until user selects 6 (use a do-while loop for the menu). Problem

Java Implementation:

Main.java

```

/*
Problem 15: Menu-Driven Calculator Display a menu:
1. Addition
2. Subtraction

```

3. Multiplication
4. Division
5. Modulus
6. Exit

Use a switch-case structure. For choice 1-5, accept two numbers and perform the operation. Loop until user selects 6 (use a do-while loop for the menu).

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        byte key;

        do {
            System.out.println("""
                \n--- Menu-Driven Calculator ---
                1. Addition
                2. Subtraction
                3. Multiplication
                4. Division
                5. Modulus
                6. Exit
                """);
            System.out.print("Enter your choice: ");
            key = input.nextByte();

            if (key == 6) {
                System.out.println("Exiting... Thank you!");
                break;
            }

            if (key < 1 || key > 6) {
                System.out.println("Invalid choice! Please try again.");
                continue;
            }

            System.out.print("Enter two numbers: ");
            float n1 = input.nextFloat();
            float n2 = input.nextFloat();
            float result = 0;
            char op = ' ';

            switch (key) {
                case 1 -> { result = n1 + n2; op = '+'; }
                case 2 -> { result = n1 - n2; op = '-'; }
                case 3 -> { result = n1 * n2; op = '*'; }
                case 4 -> {
                    if (n2 == 0) {
                        System.out.println("Error: Division by zero!");
                        continue;
                    }
                }
            }
        } while (key != 6);
    }
}
```



```

        result = n1 / n2; op = '/';
    }
    case 5 -> {
        if (n2 == 0) {
            System.out.println("Error: Modulus by zero!");
            continue;
        }
        result = n1 % n2; op = '%';
    }
}

System.out.printf("Result: %.2f %c %.2f = %.2f%n", n1, op,

    } while (key != 6);
}
}

```

Days in Month Calculator

Question Description:

m 16: Days in Month Calculator Accept month number (1-12) and year. Use switch-case to determine the number of days in that month. For February, check if it's a leap year to return 28 or 29 days. Proble

Java Implementation:

Main.java

```

/*
Problem 16: Days in Month Calculator
    Accept month number (1-12) and year.
    Use switch-case to determine the number of days in that month.
    **** For February, check if it's a leap year to return 28 or 29 day
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.println("--- Days in Month Calculator ---");

        String[] months = {"January", "February", "March", "April", "Ma
            "July", "August", "September", "October", "November", "

        System.out.print("Enter Month(1-12): ");
        byte month = input.nextByte();

        System.out.print("Enter Year: ");
        short year = input.nextShort();
    }
}

```

```

int days = switch(month){
    case 1, 3, 5, 8, 10, 12 -> 31;
    case 4, 6, 9, 11 -> 30;
    case 2 -> {
        if( ( (year % 4 == 0) && (year % 100 != 0) ) || (year %
            yield 29;
        else
            yield 28;
        }
    default -> 0;
};

if( month < 1 || month > 12){
    System.out.println("Invalid Month Number");
}
else{
    // To Prevent `IndexOutOfBoundsException` -> use [index - 1
    System.out.printf("Month %s in year %d has %d days.%n", mon
}
}
}

```

Largest of Three

Question Description:

m 17: Largest of Three (Ternary) Find the largest of three numbers using only nested ternary operators (no if statements). Print the largest number with a message identifying which variable it was. Problem

Java Implementation:

Main.java

```

import java.util.Scanner;

/*
Problem 17: Largest of Three (Ternary) Find the largest of three number
ternary operators (no if statements). Print the largest number with a m
variable it was.
*/
public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Enter 3 Numbers: ");
        short n1 = input.nextShort();
        short n2 = input.nextShort();
        short n3 = input.nextShort();

        // Nested Ternary Operator -> Condition ? (Nested Condition) :

```

```

        short largest = (n1 > n2)? (n1 > n3)? n1 : n3 : (n2 > n3)? n2 :
        System.out.printf("Largest Number: %d\n", largest);

        // Another Solution
        String result = (n1 >= n2 && n1 >= n3) ? "n1 is the largest ("
            (n2 >= n1 && n2 >= n3) ? "n2 is the largest ("
                "n3 is the largest ("

        System.out.printf( result);

    }
}

```

Vowel or Consonant Checker

Question Description:

m 18: Vowel or Consonant Checker Accept a single character. Use if-else to determine if it's: ☐ Vowel (a, e, i, o, u - both cases) ☐ Consonant ☐ Digit ☐ Special Character Use Character wrapper class methods (isDigit(), isLetter()) where appropriate. Problem

Java Implementation:

Main.java

```

/*
Problem 18: Vowel or Consonant Checker Accept a single character.
Use if-else to determine if it's:
    Vowel (a, e, i, o, u- both cases (UPPER, lower)
    Consonant
    Digit
    Special Character Use Character wrapper class methods (isDigit(),
*/

import java.util.Locale;
import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        // Accept a Character then convert it to lowercase.
        System.out.print("Enter a Character: ");
        char symbol = input.next().charAt(0);

        // Character Wrapper Class
        symbol = Character.toLowerCase(symbol);

        if (Character.isDigit(symbol)) {
            System.out.println("Digit (Number): " + symbol);

```

```

    }
    else if (Character.isLetter(symbol)) {
        if (symbol == 'a' || symbol == 'e' || symbol == 'i' || symbol == 'o' || symbol == 'u') {
            System.out.println("Vowel Letter: " + symbol);
        }
        else {
            System.out.println("Consonant Letter: " + symbol);
        }
    }
    else {
        System.out.println("Special Character: " + symbol);
    }
}
}

```

Tax Calculator

Question Description:

Java Implementation:

Main.java

```

import java.util.Scanner;

/*
Problem 19: Tax Calculator Calculate income tax based on these slabs:
    0 -> 2,50,000: No tax
    2,50,001 -> 5,00,000: 5%
    5,00,001 -> 10,00,000: 20%
    Above 10,00,000: 30%
    Use if-else-if ladder.
    Add 4% health and education cess on the calculated tax.
*/
public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your income: ");
        double income = input.nextDouble();
        double tax = 0;

        if (income >= 0 && income <= 250000) {
            tax = 0;
        }
        else if (income <= 500000) {
            tax = income * .05;
        }
        else if (income <= 1000000) {
            tax = income * 0.2;
        }
    }
}

```

```

        else{
            tax = income * 0.3;
        }

        double cess = tax * 0.04;
        double totalTax = tax + cess;

        System.out.printf("Tax (before cess): %.2f\n", tax);
        System.out.printf("Health & Education Cess (4%%): %.2f\n", cess);
        System.out.printf("Total Tax to be paid: %.2f\n", totalTax);
    }
}

```

Tax Calculator v2

Question Description:

m 19: Tax Calculator Calculate income tax based on these slabs: ☐ 0 - 2,50,000: No tax ☐ 2,50,001 - 5,00,000: 5% ☐ 5,00,001 - 10,00,000: 20% ☐ Above 10,00,000: 30% ☐ Use if-else-if ladder. Add 4% health and education cess on the calculated tax. SECTION D: LOOPS (Questions 20-27) Problem

Java Implementation:

Main.java

```

import java.util.Scanner;

/*
Problem 19: Tax Calculator Calculate income tax based on these slabs:
    0-2,50,000: No tax
    2,50,001- 5,00,000: 5%
    5,00,001- 10,00,000: 20%
    Above10,00,000: 30%
    Use if-else-if ladder. Add 4% health and education cess on the ca

The progressive tax system is the standard system used in most coun
In this case, we don't multiply the entire income by a single perce
The logic:
    First bracket: We subtract the first $250,000 (these are taxed
    Second bracket: We take the amount between $250,001 and $500,00
    Third bracket: We take the amount between $500,001 and $1,000,0
    Any amount exceeding this is multiplied by 30%.

*/
public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your income($): ");
        double income = input.nextDouble();

        double tax = 0;

```

```

        if(income > 1000000){
            // Fourth bracket: Tax on what exceeds one million + the fu
            tax += (income - 1000000) * 0.30;

            // Third bracket tax (from 500 thousand million)
            tax += (500000 * 0.20);

            // Second bracket tax (from 250 thousand to 500 thousand)
            tax += (250000 * 0.05);
        }
        else if(income > 500000){
            tax += (income - 500000) * 0.20;
            tax += (250000 * 0.05);
        }
        else if(income > 250000){
            tax += (income - 250000) * 0.05;
        }
        else{
            tax = 0;
        }

        // Add the 4% cess once
        double totalTax = tax * 1.04;    // tax + (tax * 0.04)
        System.out.println("Tax (before cess): " + tax);
        System.out.println("Cess: " + (tax * 0.04));
        System.out.printf("Total Progressive Tax: %.2f%n", totalTax);
    }
}

```

Multiplication Table

Question Description:

m 20: Multiplication Table Accept a number and print its multiplication table from 1 to 20 using a for loop. Format the output as: 5 x 1 = 5 5 x 2 = 10 ...
Proble

Java Implementation:

Main.java

```

/*
Problem 20: Multiplication Table Accept a number and print its multipli
using a for loop. Format the output as:
5 x 1 = 5
5 x 2 = 10
...
5 x 20 = 100
*/

import java.util.Scanner;

```

```

public class Main{
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a Number: ");
        int number = input.nextInt();

        System.out.println("Multiplication Table");
        System.out.println("-----");
        for(int i = 1; i <= 20; i++){
            System.out.printf("%d x %-2d = %-3d\n", number, i, (number
                // System.out.println(number + " x " + i + " = " + (number
            }

        // %-2d -> means reserve space for two digits and align to the
        // %-3d -> means reserve space for three digits so the result c

    }
}

```

Factorial Calculator

Question Description:

m 21: Factorial Calculator Calculate the factorial of a given number using: ☐
 Method A: for loop ☐ Method B: while loop ☐ Handle negative inputs
 (factorial not defined) and $0! = 1$. Use long data type for the result. Problem

Java Implementation:

Main.java

```

import java.util.Scanner;

/*
Problem 21: Factorial Calculator Calculate the factorial of a given num
    Method A: for loop
    Method B: while loop
    Handle negative inputs (factorial not defined) and  $0! = 1$ . Use lo
the result.
*/
public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a Number: ");
        int number = input.nextInt();

        if(number >= 0){
            System.out.printf("(For Method) -> The Factorial of %d (%d!
            System.out.printf("(While Method) -> The Factorial of %d (%

```

```

    }
    //      else if(number == 0){
    //          System.out.printf("The Factorial of %d (%d!) = %d%n", num
    //      }
    else{
        // Negative Number
        System.out.println("Cannot compute factorial for negative n
    }
}

public static long calcFactorialFor(int number){
    long f = 1;
    for(int i = 2; i <= number; i++){
        f = f * i;
    }
    return f;
}

public static long calcFactorialWhile(int number){
    long f = 1;
    int i = 2;
    while( i <= number ){
        f = f * i;
        i++;
    }

    return f;
}
}

```

Fibonacci Series

Question Description:

Java Implementation:

Main.java

```

/*
Problem 22: Fibonacci Series
    Print the first n terms of the Fibonacci sequence (0, 1, 1, 2, 3, 5
    where n is input by the user.
    Use a for loop. Ensure you handle the first two terms separately.
    --> Each number is the sum of the two numbers preceding it.
    --> Mathematical Equation:  $F(n) = F(n-1) + F(n-2)$  , where  $F(0) = 0$ 
*/
import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);
    }
}

```



```

        System.out.print("Enter a Number: ");
        int n = input.nextInt();

        // Recursion Solution
        System.out.println("Result = " + (calcFibonacciSeries(n)));
    }

    public static long calcFibonacciSeries(int number){
        if(number == 1){
            return 1;
        }
        else if(number == 0){
            return 0;
        }
        else{
            return calcFibonacciSeries( number - 1) + calcFibonacciSeries(number - 2);
        }
    }
}

```

Fibonacci Series using Loops

Question Description:

m 22: Fibonacci Series Print the first n terms of the Fibonacci sequence (0, 1, 1, 2, 3, 5, 8...) where n is input by the user. Use a for loop. Ensure you handle the first two terms separately. Problem

Java Implementation:

Main.java

```
import java.util.Scanner;
```

```
/*
```

Problem 22: Fibonacci Series

Print the first n terms of the Fibonacci sequence (0, 1, 1, 2, 3, 5, 8...) where n is input by the user.

Use a for loop. Ensure you handle the first two terms separately.

--> Each number is the sum of the two numbers preceding it.

```
*/
```

```

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Enter the number of terms (n): ");
        int n = input.nextInt();
    }
}

```

```

        if (n <= 0) {
            System.out.println("Please enter a positive number.");
        } else {
            System.out.println("Fibonacci Series up to " + n + " terms:");
            printFibonacci(n);
        }
    }

    public static void printFibonacci(int n){
        long firstTerm = 0, secondTerm = 1;

        for(int i = 1; i <= n; i++){
            // Display the Current Number
            System.out.print(firstTerm + ((i < n)? ", " : "\n"));

            // Compute the Next Term
            long nextTerm = firstTerm + secondTerm;

            // Update terms for next step
            firstTerm = secondTerm;
            secondTerm = nextTerm;
        }
    }
}

```

Prime Number Checker

Question Description:

m 23: Prime Number Checker Accept a number and determine if it's prime using a loop that runs from 2 to $\sqrt{n}$ (use `Math.sqrt()`). Optimize the loop to skip even numbers after checking 2. Problem

Java Implementation:

Main.java

```

import java.util.Scanner;

/*
Problem 23: Prime Number Checker
    Accept a number and determine if it's prime using a loop that runs
    Optimize the loop to skip even numbers after checking 2

    --> Hint: A prime number is a number that is only divisible by itself
    This means you'll need a for loop starting from 2 and going to the
*/
import java.util.Scanner;

public class Main {
    public static void main(String[] args){

```

```

Scanner input = new Scanner(System.in);

System.out.print("Enter a number: ");
int n = input.nextInt();

boolean isPrime = true; // افترضنا إنه صح وغيرناه لو ثبت العكس

if(n <= 1)
    isPrime = false;
else if(n == 2) // اسأل عن ال 2 الأول
    isPrime = true;
else if( n % 2 == 0) // false أي رقم زوجي ثاني (زي 4، 6، 8) هيبقى
    isPrime = false;
else {
    for(int i = 3; i <= Math.sqrt(n); i += 2) {
        if(n % i == 0) {
            isPrime = false;
            break;
        }
    }
}

if (isPrime) {
    System.out.println(n + " is a Prime Number 🍀");
} else {
    System.out.println(n + " is NOT a Prime Number ✖");
}
}

```

NumberPyramid

Question Description:

m 24: Number Pyramid Using nested for loops, print this pattern for n=5: 1 1
2 1 2 3 1 2 3 4 1 2 3 4 5 Problem

Java Implementation:

Main.java

```

/*
Problem24: Number Pyramid Using nested for loops,
print this pattern for n=5: (Centered Pyramid)
    ____1
    ____1 2
    ____1 2 3
    ____1 2 3 4
    ____1 2 3 4 5
*/
public class Main {
    public static void main(String[] args){

```

```

byte n = 5;

for(int i=1; i<=n; i++) {           // Number of Lines printed

    for (int k = (n-i); k >= 1; k--) {      // Space Loop
        System.out.print(" ");
    }
    for (int j = 1; j <= i; j++) {    // The numbers itself
        System.out.printf("%d ", j);
    }
    System.out.println();
}

}

```

Do-While Input Validation

Question Description:

m 25: Do-While Input Validation Write a program that repeatedly asks the user to enter a positive even number until they comply. Use a do-while loop to ensure the prompt appears at least once. (Simulate user input using a predefined array for testing). Problem

Java Implementation:

Main.java

```

/*
Problem25:Do-While Input Validation
Write a program that repeatedly asks the user to enter a positive even
Use a do-while loop to ensure the prompt appears at least once.
(Simulate user input using a predefined array for testing).

Your task:
Use a do-while loop to "ensure" that the program asks the user at least
Keep the loop running (looping) and repeat the question as long as (whi
As soon as the user enters a "correct" number (meaning both even and po
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);
        byte number;

        do{
            System.out.print("Enter even positive number: ");
            number = input.nextByte();

```

```

        if(number <= 0)          // Consider 0 is Even but not positive
            System.out.println("Negative Number -> Try again.");
        else if(number % 2 != 0)
            System.out.println("Odd Number -> Try again.");

        }while(number % 2 != 0 || number < 0);    // Repeat when number

        System.out.println("Success! " + number + " is an even positive
    }
}

```

EnhancedForLoopSum

Question Description:

m 26: Enhanced For Loop Sum Initialize an int array with 10 values. Use an enhanced for-loop (for-each) to calculate and display: ☐ Sum of all elements ☐ Sum of even numbers only ☐ Average as a double Problem

Java Implementation:

Main.java

```

/*
Problem26:Enhanced For Loop Sum
- Initialize an int array with 10 values.
- Use an enhanced for-loop(for-each) to calculate and display:
    Sum of all elements
    Sum of even numbers only
    Average as a double
*/
public class Main {
    public static void main(String[] args){
        // Reserve 10 integer numbers in memory consecutively
        int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

        int sumTotal = 0, sumEven = 0;
        double average = 0.0d;

        // For each int named `n` in `numbers` array
        for(int n: numbers){
            // Here n represents the current number in each iteration
            sumTotal += n;

            if(n % 2 == 0)
                sumEven += n;
        }
        average = (double) sumTotal / numbers.length;

        System.out.println("Total Sum = " + sumTotal);
        System.out.println("Total Even Sum = " + sumEven);
        System.out.println("Average = " + average);
    }
}

```

```
}  
}
```

ReverseNumber

Question Description:

Java Implementation:

Main.java

```
import java.util.Scanner;  
  
/*  
Problem27:Reverse Number  
- Accept an integer and print its reverse using a while loop  
- (e.g., input 12345 → output 54321).  
- Handle negative numbers (preserve the sign).  
  
- The remainder of a negative number (for example -123 % 10) comes out  
  - the sign will stay with you correctly and the result will automat  
*/  
public class Main {  
    public static void main(String[] args){  
        Scanner input = new Scanner(System.in);  
  
        System.out.print("Enter a Number: ");  
        int number = input.nextInt();  
        int digit = 0;  
        int reversed = 0;  
  
        //      System.out.println(12345 % 10);      // 5  
        //      System.out.println(12345 % 100);     // 45  
        //      System.out.println(12345 % 1000);    // 345  
        //      System.out.println(12345 % 10000);   // 2345  
        //      System.out.println(12345 % 100000);  // 12345  
  
        //      12345 % 10 -> 5 , 12345 / 10 -> 1234  
        //      1234 % 10 -> 4 , 1234 / 10 -> 123  
        //      123 % 10 -> 3 , 123 / 10 -> 12  
        //      12 % 10 -> 2 , 12 / 10 -> 1  
        //      1 % 10 -> 1 , 1 / 10 -> 0  
  
        //      5 * 10 = 50  
        //      + 4 = 54  
        //      54 * 10 = 540  
        //      + 3 = 543  
        //      543 * 10 = 5430  
        //      + 2 = 5432  
        //      5432 * 10 = 54320
```

```
//          + 1 = 54321

while(number != 0){
    digit = number % 10;
    number = number / 10;
    reversed = (reversed * 10) + digit;
}

System.out.println(reversed);
}
}
```

The Reverse Printer

Question Description:

m 27: Reverse Number Accept an integer and print its reverse using a while loop (e.g., input 12345 →output 54321). Handle negative numbers (preserve the sign). SECTION E: ARRAYS & STRINGS (Questions 28-32) Problem

Java Implementation:

Main.java

```
import java.util.Scanner;

/*
Problem 15: The Reverse Printer

    Ask the user to enter 5 grades.

    Storage them in a matrix.

    Print the grades exactly as they appear (from beginning to end).

    Print the grades in reverse order (from fifth to first).
*/
public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.println("Enter your 5 Marks: ");

        // Initialize Array with 5 Integers
        int[] marks = new int[5];

        // Accept 5 Marks from the user
        for(int i = 0; i < marks.length; i++){
            marks[i] = input.nextInt();
        }

        // Display Original Order
```

```

        System.out.print("Original Order: ");
        for (int i = 0; i < marks.length; i++) {
            System.out.print(marks[i] + (i < marks.length - 1 ? ", " :
        }

        System.out.println();

        // Display Reverse Order
        System.out.print("Reverse Order: ");
        for(int i = marks.length - 1; i >= 0; i--){
            System.out.print(marks[i] + (i > 0? ", " : ""));
        }
    }
}

```

ArrayStatistics

Question Description:

m 28: Array Statistics Create an array of 20 integers initialized with random values (use Math.random()). Write methods to find: ☐ Maximum value ☐ Minimum value ☐ Second largest value ☐ Sum of all elements using enhanced for-loop Problem

Java Implementation:

Main.java

```

/*
Problem28:Array Statistics
- Create an array of 20 integers initialized with random values (use Mat
- Write methods to find:
    Maximum value
    Minimum value
    Second largest value
    Sum of all elements using enhanced for-loop
*/
public class Main {
    public static void main(String[] args){
        int[] numbers = new int[20];

        for(int i = 0; i < numbers.length; i++) {
            // As Math.random() : returns a number between 0 and 1 (ex
            numbers[i] = (int) (Math.random() * 10);
        }

        displayArray(numbers);
        System.out.println("Maximum Value: " + findMax(numbers));
        System.out.println("Minimum Value: " + findMin(numbers));
        System.out.println("Second Largest Value: " + findSecondLargest
        System.out.println("Total Sum: " + calcSum(numbers));
    }
}

```



```

    }

    static void displayArray(int[] arr){
        for(int i = 0; i < arr.length; i++){
            System.out.print(arr[i] + (i == arr.length - 1 ? "\n" : " "))
        }

        static int findMax(int[] arr){
            int max = arr[0];
            for(int a: arr){
                if(a > max)
                    max = a;
            }
            return max;
        }

        static int findSecondLargest(int[] arr){
            int max = Integer.MIN_VALUE;
            int secondLargest = Integer.MIN_VALUE;
            for(int i = 0; i < arr.length; i++){
                if(arr[i] > max) {
                    secondLargest = max;
                    max = arr[i];
                }
                else if( arr[i] > secondLargest && arr[i] < max){
                    secondLargest = arr[i];
                }
            }
            return secondLargest;
        }

        static int findMin(int[] arr){
            int min = arr[0];
            for(int a: arr){
                if(a < min)
                    min = a;
            }
            return min;
        }

        static int calcSum(int[] arr){
            int sum = 0;
            for(int a: arr)
                sum += a;

            return sum;
        }
    }
}

```

StringCase Toggle - StringBuilder

Question Description:

Java Implementation:

Main.java

```
import java.util.Scanner;

/*
- The StringBuilder is the programmer's loyal friend when dealing with
  because it saves memory consumption and makes the code much faster.
- Imagine a regular String as if it were a 'paper written with ink';
  if you want to change it, you have to tear the paper and get a new
- But the StringBuilder is like a 'whiteboard'; you can erase and add f
*/

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a Text: ");
        String text = input.nextLine();

        // Instantiate an Object
        StringBuilder result = new StringBuilder();

        for(int i = 0; i < text.length(); i++){
            char currentChar = text.charAt(i);

            if(i % 2 == 0)
                result.append(Character.toUpperCase(currentChar));
            else
                result.append(Character.toLowerCase(currentChar));
        }

        System.out.println("Converted Text: " + result.toString());
    }

    /*
    Why this version is 'smarter'?
    1) Memory Efficiency: StringBuilder does not make new copies of the
    2) Speed: For long texts, StringBuilder is thousands of times faster
    */
}
```

StringCaseToggle

Question Description:

m 29: String Case Toggle Accept a String and create a new String where every

alternate character is uppercase and lowercase (e.g., "hello" → "HeLlO"). Use charAt() and String concatenation in a loop. Problem

Java Implementation:

Main.java

```
import java.util.Scanner;

/*
Problem29: StringCase Toggle
- Accept a String and create a new String where every alternate character
- (e.g., "hello" → "HeLlO").
- Use charAt() and String concatenation in a loop.
*/
public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a Text: ");
        String text = input.next();    // Immutable -> Not adjustable
        String result = "";

        // As String = Array of Characters -> We can loop over it
        for(int i = 0; i < text.length(); i++){
            if( i % 2 != 0){    // ODD -> LowerCase
                result += Character.toLowerCase( text.charAt(i) );
            }
            else{    // EVEN -> UpperCase
                result += Character.toUpperCase( text.charAt(i) );
            }
        }

        System.out.println("Original Text: " + text);
        System.out.println("Converted Text: " + result);
    }
}
```

Autoboxing-Unboxing Demonstration

Question Description:

m 30: Autoboxing/Unboxing Demonstration Create a method that accepts a primitive int array and returns the sum as an Integer object (autoboxing). In main, unbox this Integer to a primitive int and display it. Also demonstrate converting an int to String using wrapper class. Problem

Java Implementation:

Main.java

```
/*
Problem30: Autoboxing/Unboxing Demonstration
```

- Create a method that accepts a primitive int array and returns the sum as an Integer object (autoboxing).
- In main, unbox this Integer to a primitive int and display it.
- Also demonstrate converting an int to String using wrapper class.

Autoboxing and Unboxing

This is the 'automatic conversion' that Java does between a Primitive a

1. Autoboxing: It is converting a Primitive to a Wrapper automatically.
2. Unboxing: It is the opposite, converting a Wrapper Object to a P

```

*/
public class Main {
    public static void main(String[] args){
        int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

        // Unboxing
        int sum = calcSum(numbers);

        String sumString = Integer.toString(sum);

        System.out.println("The sum is: " + sumString);
    }

    static Integer calcSum(int[] arr){
        // Return Type: Integer for Autoboxing
        int sum = 0;
        for(int a: arr)
            sum += a;

        return sum;
    }
}

```

Linear Search

Question Description:

Java Implementation:

Main.java

```

/*
Problem 31: Linear Search Implement linear search on an int array. Return the element if found, otherwise, return -1. Use a for-each loop with a break statement when found.
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){

```

```

Scanner input = new Scanner(System.in);

System.out.print("Enter Array Size: ");
final int SIZE = input.nextInt();

int[] numbers = new int[SIZE];
for(int i = 0; i < SIZE; i++){
    numbers[i] = (int) (Math.random() * 10);
}

for(int i = 0; i < SIZE; i++){
    System.out.print(numbers[i] + ( (i == numbers.length - 1)?
}

System.out.print("Enter the value you search for: ");
int value = input.nextInt();

System.out.println((search(numbers, value) == -1 ? "Not Found"
}

static int search(int[] arr, int value){
    boolean found = false;
    byte counter = -1;
    for(int a: arr){
        counter++;
        if(value == a){
            found = true;
            break;
        }
    }

    if(found)
        return counter;        // The Actual Index
    else
        return -1;
}
}

```

Linear Search - Return Array

Question Description:

m 31: Linear Search Implement linear search on an int array. Return the index if found, -1 otherwise. Use a for-each loop with a break statement when found. Proble

Java Implementation:

Main.java

```
import java.util.Scanner;
```

```

/*
Like Normal Linear Search but handles the duplicates
Problem 32: Find Duplicates
- Initialize an array with random values.
- Use nested loops to compare each element with the rest of the array.
- Print the duplicate values found.
*/
public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter Array Size: ");
        int size = input.nextInt();
        int[] numbers = new int[size];

        // 1. 9 ل 0 عشوائية بأرقام المصفوفة
        System.out.print("Array Elements: ");
        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = (int) (Math.random() * 10);
            System.out.print(numbers[i] + (i == numbers.length - 1 ? " "
        }
        System.out.println("\n-----");

        // 2. استدعاء ميثود البحث عن المكررات
        findDuplicates(numbers);
    }

    static void findDuplicates(int[] arr) {
        System.out.println("Duplicate Elements:");
        boolean foundAny = false;

        // اللوب الخارجية تمسك عنصر عنصر
        for (int i = 0; i < arr.length; i++) {

            // تكة احترافية: نتحقق إذا كان الرقم ده تم فحصه قبل كدة عشان ميكررش الطباعة
            boolean alreadyPrinted = false;
            for (int k = 0; k < i; k++) {
                if (arr[k] == arr[i]) {
                    alreadyPrinted = true;
                    break;
                }
            }
            if (alreadyPrinted) continue;

            // بالعناصر اللي بعده فقط (i) اللوب الداخلية تقارن العنصر الحالي
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    System.out.println("Number " + arr[i] + " is duplic
                    foundAny = true;
                    break; // نخرج من اللوب الداخلية بمجرد إيجاد أول تكرار لهذا الرقم
                }
            }
        }
    }
}

```

```

    }

    if (!foundAny) {
        System.out.println("No duplicates found.");
    }
}
}

```

Jagged Array Sum

Question Description:

m 32: Jagged Array Sum Create a jagged array (array of arrays with different lengths) representing student marks in different subjects: □ Student 1: 3 subjects □ Student 2: 4 subjects □ Student 3: 2 subjects □ Calculate and display the average marks for each student using nested loops. SECTION F: CONTROL FLOW & JUMP STATEMENTS (Questions 33-

1.

Proble

Java Implementation:

Main.java

```

/*
Problem 32: Jagged Array Sum Create a jagged array (array of arrays wit
representing student marks in different subjects:
    Student 1: 3 subjects
    Student 2: 4 subjects
    Student 3: 2 subjects
    Calculate and display the average marks for each student using ne
-----
What is a Jagged Array?
    - it is an "array of arrays,"
    - but the difference between it and a regular 2D Array (the uniform
        - A regular 2D Array: is like a table, with all rows, for examp
        - A Jagged Array: the first row might have 3 slots, the second
*/

import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        int sum = 0;

        System.out.print("Enter Number of Students: ");

```

```

final int STUDENTS = input.nextInt();

int[][] studentMarks = new int[STUDENTS][]; // // We have (STUD

// Define Number of Subjects for each Student.
for(int i = 0; i < STUDENTS; i++){
    System.out.printf("Enter Number of Subject for %dth Student
    int Subjects = input.nextInt();

    studentMarks[i] = new int[Subjects];
}

// Insert Student Marks and Calc. Sum and Average.
for(int i = 0; i < STUDENTS; i++){
    sum = 0;
    for(int j = 0; j < studentMarks[i].length; j++){
        System.out.printf("Student %d , Enter Course %d Mark: "
        studentMarks[i][j] = input.nextInt();

        sum += studentMarks[i][j];
    }
    double average = (double) sum / studentMarks[i].length;
    System.out.printf("--- Student %d Statistics ---\n", (i + 1
    System.out.printf("        Student %d: Subject Marks: %s -> %d
    System.out.printf("        Student %d 's Total Marks sum = %d%
    System.out.println("        Average = " + average);
    System.out.println("-----");
}
}
}

```

Continue Skip Pattern

Question Description:

m 33: Continue Skip Pattern Print numbers from 1 to 50, but skip all multiples of 3 using the continue statement. Count how many numbers were printed and display the count. Proble

Java Implementation:

Main.java

```

/*
Problem 33: Continue Skip Pattern Print numbers from 1 to 50, but skip
the continue statement. Count how many numbers were printed and display
*/
public class Main {
    public static void main(String[] args){
        int counter = 0;
        for(int i = 1; i <= 50; i++){

```



```

        if(i % 3 == 0)
            continue;

        System.out.println("Number: " + i);
        counter++;
    }

    System.out.println("How Many numbers were printed? " + counter
}

```

Labeled Break Search

Question Description:

m 34: Labeled Break Search Create a 5x5 matrix (2D array) with numbers 1-25. Search for a specific number using nested loops. If found, use a labeled break to exit both loops immediately and print the position (row, col). Problem

Java Implementation:

Main.java

```

/*
Problem 34: Labeled Break Search
- Create a 5x5 matrix (2D array) with numbers 1-25.
- Search for a specific number using nested loops.
  - If found, use a labeled break to exit both loops immediately and
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        int[][] matrix = new int[5][5];
        for(int i = 0; i < matrix.length; i++){
            for(int j = 0; j < matrix[i].length; j++){
                // (Math.random() * Max) + Min
                matrix[i][j] = (int) (Math.random() * 25 + 1);
            }
        }

        System.out.print("Enter the value to search for: ");
        int value = input.nextInt();
        boolean found = false;

        search:
        for(int i = 0; i < matrix.length; i++){
            for(int j = 0; j < matrix[i].length; j++){
                if(matrix[i][j] == value){

```

```

        System.out.printf("Found at Position: Row = %d , Co
        found = true;
        break search;
    }
}

if(!found)
    System.out.println("Value Not Found.");

    System.out.println("----- Matrix -----");
    display2DArray(matrix);
}

static void display2DArray(int[][] arr) {
    for (int[] ints : arr) {
        for (int anInt : ints) {
            System.out.print(anInt + " ");
        }
        System.out.println();
    }
}
}

```

Return Statement Usage

Question Description:

m 35: Return Statement Usage Write a method `isPerfectNumber(int n)` that returns boolean (use return statement). A perfect number equals the sum of its proper divisors (e.g., $6 = 1 + 2 + 3$). In main, use this method to print all perfect numbers between 1 and 1000. Demonstrate early return if number ≤ 0 .

Java Implementation:

Main.java

```

/*
Problem 35: Return Statement Usage
- Write a method isPerfectNumber(int n) that returns boolean (use retur
- A perfect number equals the sum of its proper divisors (e.g.,  $6 = 1+2$ 
- In main, use this method to print all perfect numbers between 1 and 1
- Demonstrate early return if number  $\leq 0$ .
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

```

```

        // System.out.print("Enter a Number: ");
        // int n = input.nextInt();
        for(int i = 1; i <= 1000; i++){
            if( isPerfectNumber(i) )
                System.out.println(i + " Perfect Number.");
            // else
            // System.out.println(i + " Not Perfect Number.");
        }
    }

    static boolean isPerfectNumber(int n){
        if(n <= 0)
            return false;

        int sum = 0;
        for(int i = (n - 1); i > 0; i--){
            if(n % i == 0)
                sum += i;
        }

        return (sum == n);
    }
}

// 6 -> 5, 4, 3, 2, 1
// loop (for int i = 6; i > 0; i--)
// if(6 % i) == 0 -> Save it in array --> But the Problem: Size is not
// Instead : use variable to sum all these divisors (on the fly).
// At the End -> Check this sum == 6

```

Compound Interest Calculator

Question Description:

(Question text not found in PDF. This might be an extra practice or alternative solution.)

Java Implementation:

Main.java

```

/*
Problem 6: (((Compound Interest))) Calculator Calculate Compound intere
Accept Principal(P) (long), Rate(r) (float), and Time(t) (int) using ap
minimize memory usage. Display the result with 2 decimal places.
*/

import java.util.Scanner;

public class Main {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);
    }
}

```

```

System.out.print("Enter the Principal Value (P): ");
long principal = input.nextLong();

System.out.print("Enter the Annual Interest Rate (R%): ");
float rate = input.nextFloat();

System.out.print("Enter the Time in years (T): ");
int time = input.nextInt();

// حساب المبلغ الكلي باستخدام قانون الفائدة المركبة
// Math.pow(base, exponent) وترجع دائماً double
double totalAmount = principal * Math.pow((1 + rate / 100.0), time);

// الفائدة الربحية فقط هي المبلغ الكلي ناقص الأصلي
double compoundInterest = totalAmount - principal;

System.out.println("-----");
System.out.printf("Compound Interest Value: %.2f\n", compoundInterest);
System.out.printf("Total Balance after %d years: %.2f\n", time, totalAmount);
}
}

```

Inverted Pyramid

Question Description:

(Question text not found in PDF. This might be an extra practice or alternative solution.)

Java Implementation:

Main.java

```

/*
Display the Pyramid but inverted:
    1 2 3 4 5
   _1 2 3 4
  __1 2 3
 ___1 2
____1
*/
public class Main {
    public static void main(String[] args){
        byte n = 5;

        for(int i = n; i >= 1; i--){           // Number of Lines display
            for(int k = 1; k <= (n - i); k++){ // Spaces Loop
                System.out.print(" ");
            }

            for(int k = 1; k <= i; k++){         // Numbers itself

```

```
        System.out.printf("%d ", k);
    }
    System.out.println();
}
}
```

Max-Min-Array

Question Description:

(Question text not found in PDF. This might be an extra practice or alternative solution.)

Java Implementation:

Main.java

```
/*
Find the Maximum and Minimum Numbers in Array of 10 Integer Numbers using
*/

public class Main {
    public static void main(String[] args){
        int[] numbers = {10, -100, 1212, 2132, 43, -1213, 21312, 1, 2,
        3};

        int max = numbers[0], min = numbers[0];
        for(int n: numbers){
            if(n > max)
                max = n;

            if(n < min)
                min = n;
        }

        System.out.println("Maximum Number = " + max);
        System.out.println("Minimum Number = " + min);
    }
}
```
